

SentinAL: Secure AI Desktop Automation

A Major Project report

Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING (AI&ML)

Submitted by

Gannoju pavan kumar 23BT5A6601

Eeranki Meghana 22BT1A6619

P Karthik 22BT1A6605

Lenkala Shiva 22BT1A6636

Under the Guidance of
V. Hirish Reddy
Assistant Professor



VISVESVARAYA COLLEGE OF ENGINEERING & TECHNOLOGY

(Recognized by A.I.C.T.E Affiliated to JNTU, Hyderabad),
(MP Patelguda, Ibrahimpatnam, Telangana, INDIA - 501510)

2025- 2026



VISVESVARAYA

COLLEGE OF ENGINEERING & TECHNOLOGY



Approved by AICTE, New Delhi & Govt. of T.S. Accredited with NAAC 'A' Grade, Affiliated to JNTUH, Hyderabad
Sponsored by : Jawahar Educational Society, An ISO 9001 : 2018 and ISO 14001 : 2015 Certified Institution

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI&ML)

CERTIFICATE

This is to certify that this project report entitled SentinAL: Secure AI Desktop Automation submitted by **Gannaju Pavan (23BT5A6601)**, **Eeranki Meghana (22BT1A6619)**, **P Karthik (22BT1A6605)**, **Lenkala Shiva (22BT1A6636)**, in partial fulfillment of the requirements for the degree of **Bachelor of Technology in Computer Science & Engineering(AI&ML)** to the Jawaharlal Nehru Technological University, Hyderabad, during the academic year 2025 - 26, is a bonafide record of work carried out under our guidance and supervision.

The results embodied in this report have not been submitted to any other University or Institution for the award of any degree or diploma.

Mr.V.Hirish Reddy
Assistant Professor
Internal Guide

Mr.V.Hirish Reddy
Head of Department
Dept. of CSE (AI&ML)

External Examiner

DECLARATION

I hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signature:

Name: Gannoju pavan

Eeranki Meghana

P Karthik

Lenkala Shiva

Roll No: 23BT5A6601

22BT1A6619

22BT1A6605

22BT1A6636

ACKNOWLEDGEMENT

It gives us a great sense of pleasure to present the report of the project undertaken during B.Tech. We would like to express our special thanks to our Principal **Dr.D.Ramesh** for moral support and **college Management** of Visvesvaraya College of Engineering & Technology, Hyderabad for providing us infrastructure to complete the project.

We owe special debt of gratitude to **Dr.K.V.Ram Prasad, Dean Academics** of Visvesvaraya College of Engineering & Technology, Hyderabad for his constant support and guidance throughout the course of our work.

We thank **Mr.V.Hirish Reddy** Head of the Department of Computer Science & Engineering (AI&ML) for his constant support and cooperation.

We owe special debt of gratitude to our Guide Dr.Syed Thaisin, Assistant Professor Visvesvaraya College of Engineering & Technology, Hyderabad for her guidance throughout the course of our work. It is only her cognizant efforts that our endeavors have seen the light of the day.

We also like to express our gratitude towards our **Parents/Guardians** & siblings for their kind co-operation and encouragement which helped us in completion of this project.

We do not want to miss the opportunity to acknowledge the contribution of all faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our **friends** for their contribution in the completion of the project.

Gannaju Pavan, 23BT5A6601

Eeranki Meghana, 22BT1A6619

P Karthik, 22BT1A6605

Lenkala Shiva, 22BT1A6636

ABSTRACT.....	1
1.0 INTRODUCTION.....	2
1.1 Domain Description.....	3
1.2 Problem Definition.....	3
1.3 Proposed Solutions.....	4
1.4 Objectives.....	4
2.0 LITERATURE SURVEY.....	5
2.1 Existing System.....	5
2.2 proposed System.....	6
2.3 Advantages of Proposed System.....	6
2.4 Feasibility Study.....	7
3.0 SYSTEM ANALYSIS.....	7
3.1 Specifications.....	8
Functional Specifications:.....	8
Non-Functional Specifications:.....	8
3.2 Software Requirements.....	8
3.3 Hardware Requirements.....	9
3.4 System Architecture.....	9
3.5 System Architecture Diagram.....	9
3.6 Module Description.....	10
3.7 Megha SubSystem.....	11
4.0 DESIGN.....	11
4.1 Block Diagram.....	12
4.2 Data Flow Diagram (DFD).....	12
4.2.1 DFD Symbols.....	13
4.2.2 Context Level Diagram.....	13
4.2.3 Level 1 DFD.....	14
4.3 Unified Modeling Language (UML).....	15
4.3.1 Class Diagram.....	15
4.3.2 Use Case Diagram.....	16
4.3.3 Sequence Diagram.....	17
4.3.4 Activity Diagram.....	18
5.0 IMPLEMENTATION.....	21
5.1 Overview.....	21
5.2 Tools and Technologies Used.....	21
5.3 System Architecture Implementation.....	22
5.4 Module Implementation.....	22
5.5 Workflow Execution.....	26
5.6 Key Features Implemented.....	26
6.0 TESTING.....	27

6.1 INTRODUCTION.....	27
6.1.1 Unit Testing.....	27
6.1.2 Integration Testing.....	27
6.1.3 System Testing (Black-Box).....	28
6.2 Design Of Test Cases and Scenarios.....	28
6.3 Input Testing.....	29
6.4 Execution Testing.....	29
6.5 Error Handling.....	29
6.6 Performance Testing.....	30
6.7 Test Results Summary.....	30
7.0 OUTPUT SCREENS.....	30
7.1 System Initialization and Dashboard.....	30
7.2 Successful Multi-Step Cloud Execution.....	31
7.3 Privacy-Routed Local Execution.....	32
7.4 SentinAL Security Blocklist Interception.....	32
7.5 React Self-Healing Loop Recovery.....	32
8.0 CONCLUSION.....	33
9.0 FUTURE ENHANCEMENT.....	33
10.0 REFERENCES.....	34

ABSTRACT

Traditional desktop operating systems rely on rigid graphical user interfaces and strict command-line syntax, which often results in inflexible workflows and requires precise manual inputs. Additionally, integrating probabilistic Large Language Models (LLMs) directly into deterministic system environments introduces challenges related to execution accuracy, security, and reliability.

This project presents SentinAL (Sentinel + AI), a hybrid artificial intelligence-powered desktop orchestration system designed to bridge the gap between natural language interaction and secure operating system execution. Unlike conventional rule-based desktop assistants, the proposed system adopts a modular seven-layer architecture that enables real-time communication through a FastAPI and WebSocket-based pipeline.

A key contribution of this system is the SentinAL Security and Governance Layer, which ensures safe operation through intent allowlisting, command blocklisting, sandbox-based validation, and continuous audit logging. To balance performance and privacy, the system incorporates a hybrid LLM routing mechanism that dynamically switches between a local offline language model and cloud-based models depending on task sensitivity and computational requirements.

To enhance execution reliability, the system implements a self-healing mechanism based on a reasoning and acting loop, which detects execution failures, analyzes error outputs, and automatically corrects commands. Additional components such as an SQLite-based memory system, offline text-to-speech module, and modular service architecture further improve system responsiveness and usability.

The proposed system demonstrates a secure, efficient, and scalable approach to intelligent desktop automation, providing a practical solution for integrating natural language interfaces with operating system control while maintaining strong safety and privacy guarantees.

1.0 INTRODUCTION

The evolution of human-computer interaction has consistently focused on improving accessibility and efficiency, enabling users to interact with computing systems in more intuitive ways. In modern desktop environments, users frequently perform repetitive and multi-step tasks such as managing files, launching applications, and navigating system interfaces. These tasks are typically executed through Graphical User Interfaces (GUIs) or Command-Line Interfaces (CLIs), both of which require precise inputs and adherence to predefined syntax. As a result, users often experience limitations in flexibility, efficiency, and productivity.

Traditional operating systems are inherently deterministic and are not designed to interpret natural human language. This creates a gap between how users naturally express their intentions and how systems process executable instructions. While recent advancements in Large Language Models (LLMs) have significantly improved natural language understanding, directly integrating these probabilistic models into operating system environments introduces challenges related to reliability, execution accuracy, and system security.

To address these challenges, this project introduces SentinAL, a hybrid AI-powered desktop orchestration system. The system provides a unified interface through which users can issue commands using natural language via text or voice. Instead of executing isolated commands, the system interprets user intent and generates structured representations of tasks, enabling multi-step automation within a single interaction.

The system is built on a modular seven-layer architecture and operates using a FastAPI and WebSocket-based communication pipeline for real-time interaction and feedback. A key feature of the system is the SentinAL Security and Governance Layer, which ensures that all actions are validated before execution. This layer enforces strict constraints through allowlisting, blocklisting, and sandbox-based validation to prevent unsafe operations.

Additionally, the system incorporates advanced mechanisms such as hybrid LLM routing, which dynamically switches between local and cloud-based models based on task sensitivity, and a self-healing execution process that detects and corrects errors during runtime. By combining intelligent natural language processing with secure and reliable execution, the proposed system aims to provide an efficient, scalable, and user-friendly solution for desktop automation.

1.1 DOMAIN DESCRIPTION

The domain of this project lies at the intersection of Artificial Intelligence, Natural Language Processing (NLP), Operating System Automation, and Security Governance. It extends into the emerging area of intelligent desktop orchestration, where AI systems are capable of directly interacting with a host machine's file system, command-line interface, and graphical user interface.

Unlike traditional NLP applications that are limited to generating text or answering queries, this domain focuses on enabling AI systems to perform real-world system-level actions. This introduces new challenges related to execution accuracy, security, and privacy. A critical requirement in this domain is maintaining a balance between intelligent decision-making and safe, deterministic system execution.

The project also operates within the paradigm of hybrid edge-cloud computing. Sensitive tasks are handled locally using offline language models to ensure data privacy, while non-sensitive operations can leverage cloud-based models for improved performance. This hybrid approach allows the system to achieve both efficiency and data security in real-time desktop automation.

1.2 PROBLEM DEFINITION

Integrating artificial intelligence with operating system-level execution introduces several key challenges that are not addressed by traditional desktop assistants:

One major issue is the gap between probabilistic AI models and deterministic operating systems. Large Language Models generate outputs based on probability, whereas operating systems require precise and exact inputs such as absolute file paths. This mismatch often leads to execution failures when incorrect paths are generated.

Another critical challenge is system security. Allowing an AI system to generate and execute commands without strict validation can lead to harmful or unintended operations, such as deleting important files or accessing restricted system areas.

Privacy is also a concern in existing solutions. Many cloud-based assistants transmit user data, including sensitive system information, to external servers, which compromises data security and user control.

Additionally, traditional automation systems lack fault tolerance. If an error occurs during execution, the system fails completely and requires manual intervention, reducing reliability and usability.

1.3 PROPOSED SOLUTION

To address these challenges, this project proposes SentinAL, a secure and intelligent desktop orchestration system designed to safely integrate natural language processing with operating system control.

The system operates using a real-time communication pipeline based on FastAPI and WebSockets, enabling continuous interaction and feedback between the user interface and the backend processing engine. User commands are interpreted and converted into structured multi-step actions that can be executed sequentially.

To resolve execution inaccuracies such as incorrect file paths, the system introduces the Memory-Enhanced Global Heuristic Architecture (M.E.G.H.A.), which maintains a pre-indexed SQLite database of valid system paths. This ensures accurate and fast retrieval of file locations during execution.

A dedicated SentinAL Security and Governance Layer is implemented to validate all actions before execution. This includes intent allowlisting, command blocklisting, and sandbox-based validation to ensure that only safe operations are performed.

The system also incorporates hybrid LLM routing, where sensitive operations are handled locally using offline models, while general tasks can utilize cloud-based models for faster processing.

To improve reliability, a self-healing execution mechanism based on a Reasoning and Acting (ReAct) loop is implemented. If an execution error occurs, the system captures the error, analyzes it, and automatically corrects the command before retrying execution.

1.4 OBJECTIVES

The primary objectives of this project are as follows:

To design and implement a modular, multi-layer desktop orchestration system capable of interpreting natural language commands and executing them in real time.

To develop a deterministic path resolution mechanism using a memory-based indexing system that eliminates execution failures caused by incorrect file paths.

To establish a secure validation framework that enforces strict control over all system-level actions through allowlisting, blocklisting, and sandbox validation.

To implement a fault-tolerant execution mechanism that can automatically detect and correct errors during runtime using a self-healing approach.

To develop a hybrid AI routing system that balances performance and privacy by dynamically switching between local and cloud-based language models.

To create a responsive and efficient system that improves user productivity by enabling multi-step task automation through a single natural language input.

2.0 LITERATURE SURVEY

The development of desktop automation and human-computer interaction has evolved significantly over time. Early systems relied on command-line interfaces, where users were required to input exact commands to perform specific tasks. While these systems offered precision, they required technical expertise and lacked flexibility. With the introduction of graphical user interfaces and scripting tools, usability improved, but automation remained limited to predefined workflows.

The advancement of Natural Language Processing (NLP) introduced a new paradigm, enabling systems to interpret user input in a more flexible and conversational manner. Recent developments in Large Language Models (LLMs) have further enhanced this capability by providing improved semantic understanding. However, applying these models directly to operating system environments introduces challenges related to execution accuracy, system security, and data privacy. Therefore, there is a need for systems that can combine intelligent language understanding with safe and reliable system execution.

2.1 EXISTING SYSTEM

Existing desktop automation systems can be categorized into command-line tools, rule-based assistants, and cloud-based AI assistants.

Command-line tools such as PowerShell and Bash provide precise control over system operations but require users to have technical knowledge and follow strict syntax. Rule-based assistants offer limited automation through predefined commands, but they fail to handle variations in natural language input effectively.

Cloud-based AI assistants provide improved language understanding but introduce several limitations. They depend on continuous internet connectivity and often transmit user data to external servers, which raises privacy concerns. Additionally, these systems lack strong validation mechanisms and may execute incorrect or unsafe commands. Another common issue is path hallucination, where AI models generate incorrect file paths, leading to execution failures due to the deterministic nature of operating systems.

2.2 PROPOSED SYSTEM

The proposed system, SentinAL, introduces a hybrid AI-based desktop orchestration framework that integrates intelligent language processing with secure system execution.

The system is built on a modular seven-layer architecture consisting of Input, Gateway, Processing, Security, Execution, Services, and Output layers. It uses FastAPI and WebSocket communication to enable real-time interaction between the user interface and the backend system.

To address privacy concerns, the system implements a hybrid LLM routing mechanism. Sensitive tasks are processed using a local offline language model, while general tasks can be handled using cloud-based models for improved performance.

To eliminate execution errors caused by incorrect file paths, the system introduces the M.E.G.H.A. (Memory-Enhanced Global Heuristic Architecture) subsystem. This module maintains a pre-indexed SQLite database of valid system paths, allowing accurate and fast retrieval during execution.

The system also incorporates a self-healing execution mechanism based on a Reasoning and Acting (ReAct) loop. When an error occurs during execution, the system captures the error, analyzes it, and automatically corrects the command before retrying.

2.3 ADVANTAGES OF PROPOSED SYSTEM

The proposed system provides several advantages over existing desktop automation solutions.

It ensures data privacy by processing sensitive information locally, reducing dependency on cloud-based systems. The hybrid routing approach improves performance while maintaining security.

The use of a memory-based path resolution system eliminates errors caused by incorrect file paths, ensuring accurate and reliable execution.

The system includes a strong security framework that validates all actions before execution, preventing unauthorized or harmful operations.

Additionally, the self-healing execution mechanism improves system reliability by automatically handling execution errors without requiring manual intervention.

2.4 FEASIBILITY STUDY

The feasibility of the proposed system is evaluated across technical, economic, and operational aspects.

From a technical perspective, the system is feasible as it is built using widely available technologies such as Python, FastAPI, WebSockets, and SQLite. Local language model execution is supported through frameworks such as Ollama, making offline processing practical.

Economically, the system is cost-effective as it primarily relies on open-source tools and minimal cloud usage. The use of lightweight databases such as SQLite eliminates the need for expensive infrastructure.

Operationally, the system is user-friendly and accessible. By enabling natural language interaction, it reduces the need for technical expertise. The real-time feedback mechanism and intuitive design improve usability, making the system suitable for everyday desktop automation tasks.

3.0 SYSTEM ANALYSIS

System analysis defines the functional and non-functional requirements of the proposed system and establishes the foundation for its design and implementation. It evaluates system behavior, performance constraints, security requirements, and interaction between components.

For SentinAL, the system analysis focuses on integrating artificial intelligence with operating system-level execution in a secure and efficient manner. The system is designed to process natural language inputs, convert them into structured commands, validate them through a security layer, and execute them reliably with real-time feedback.

This chapter provides a detailed description of system specifications, software and hardware requirements, architectural design, and module-level functionality.

3.1 SPECIFICATIONS

The system requirements are categorized into functional and non-functional specifications to define its capabilities and software quality standards.

Functional Specifications:

The system accepts both text and voice inputs, enabling natural language interaction.

It interprets user commands and converts them into structured multi-step actions.

The system validates all commands using a security layer before execution.

It supports execution of operating system tasks such as opening applications, accessing files, and retrieving information.

A memory-based path resolution mechanism is used to ensure accurate file access.

Non-Functional Specifications:

The system provides real-time response with minimal latency.

It includes fault-tolerant execution with automatic error handling.

The architecture is modular and scalable, allowing independent updates to system components.

The system ensures data privacy by supporting local processing of sensitive tasks.

3.2 SOFTWARE REQUIREMENTS

The system is developed using a modern software stack that supports asynchronous processing and real-time communication.

Operating System: Windows 10 or Windows 11

Programming Language: Python 3.10 or higher.

Backend Framework: FastAPI with Uvicorn for handling asynchronous requests and WebSocket communication.

Frontend Framework: React with Vite for building a responsive user interface.

Database: SQLite for storing system memory, logs, and indexed file paths.

Artificial Intelligence Integration: Ollama for local language model execution and cloud APIs (such as Groq or Gemini) for hybrid routing

Additional Libraries: requests, subprocess, and other Python standard libraries for system interaction

3.3 HARDWARE REQUIREMENTS

The system requires standard computing hardware capable of supporting local AI processing and real-time execution..

Processor: Intel Core i5 (or equivalent AMD processor) minimum

RAM: Minimum 8 GB (16 GB recommended for optimal model inference)

Storage: 10–20 GB of available disk space (primarily to store local model weights)

Peripherals: Standard input/output devices including Keyboard, Mouse, and Monitor.

3.4 SYSTEM ARCHITECTURE

The system follows a modular seven-layer architecture designed to separate concerns and improve scalability, security, and performance.

The Input Layer captures user commands through text or voice interfaces.

The Gateway Layer manages communication between the frontend and backend using FastAPI and WebSocket protocols.

The Processing Layer interprets user input using language models and converts it into structured intents.

The Security Layer validates all generated commands using allowlists, blocklists, and sandbox rules to ensure safe execution.

The Execution Layer performs validated operations on the operating system using controlled subprocess calls.

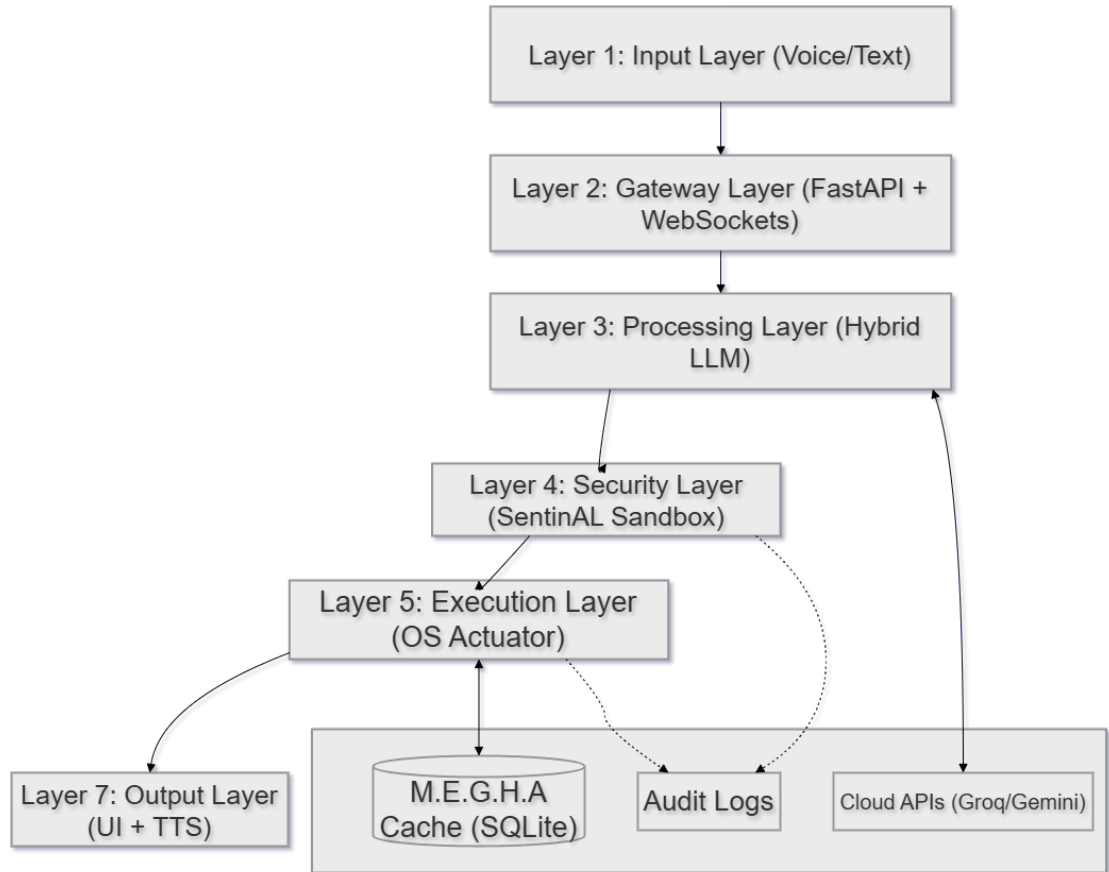
The Services Layer handles auxiliary operations such as database access, logging, and external data processing.

The Output Layer returns results to the user through text and optional audio feedback.

This layered design ensures that each component operates independently, improving system reliability and maintainability.

3.5 SYSTEM ARCHITECTURE DIAGRAM

The system architecture diagram represents the flow of data across different layers of the system.



User input is captured by the Input Layer and transmitted to the Gateway Layer. The Processing Layer analyzes the input and generates structured commands, which are then validated by the Security Layer. Once approved, the Execution Layer performs the required operations on the system. The Services Layer supports additional functionalities such as memory and logging, while the Output Layer provides feedback to the user.

This flow ensures secure, efficient, and real-time execution of user commands.

3.6 MODULE DESCRIPTION

The system is divided into multiple modules, each responsible for a specific function.

Input Module: Captures user input through text or voice interface.

Gateway Module: Handles communication between frontend and backend using WebSockets.

Processing Module: Interprets user input using language models and generates structured intents.

Security Module: Validates commands using predefined rules to prevent unsafe operations.

Execution Module: Executes validated commands on the operating system.

Services Module: Manages database operations, logging, and additional services.

Output Module: Displays results to the user through text and audio feedback.

Each module is designed to function independently, allowing easy maintenance and future enhancements.

3.7 MEGHA SUBSYSTEM

The Memory-Enhanced Global Heuristic Architecture (M.E.G.H.A.) is a specialized subsystem designed to improve execution accuracy.

Traditional AI systems often generate incorrect file paths due to their probabilistic nature. To address this issue, the system maintains a pre-indexed SQLite database containing valid file system paths.

During execution, the system retrieves accurate paths from this database instead of relying on AI-generated predictions. This ensures reliable and efficient file access, even in environments where directory structures are modified or cloud-synchronized.

The M.E.G.H.A. subsystem significantly enhances system reliability by eliminating path-related execution errors.

4.0 DESIGN

The system design of **SentinAL** defines the structure, behavior, and interaction of components within a hybrid AI-based desktop orchestration system. The architecture follows a modular and layered approach, where each component is responsible for a specific function such as input handling, communication, AI processing, security validation, execution, and service management.

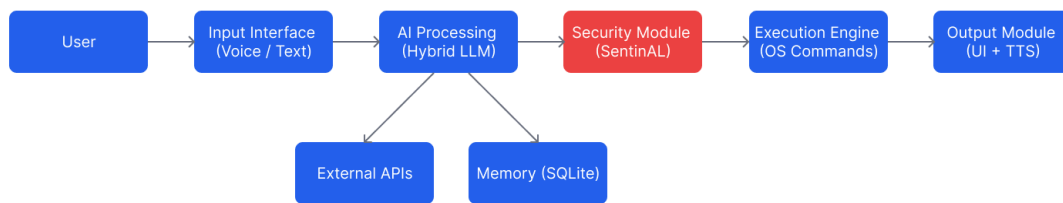
This modular design improves system scalability, maintainability, and flexibility. Each module operates independently while maintaining seamless communication with other components. Changes in one module do not significantly affect the rest of the system, enabling efficient development and testing.

The primary objective of the system design is to transform natural language user input into secure and precise system-level actions. The system integrates **hybrid Large Language**

Models (LLMs) (local and cloud), a **security validation layer (SentinAL Sandbox)**, and a **self-healing execution mechanism (ReAct loop)**. Additionally, it incorporates persistent memory using **SQLite (M.E.G.H.A)** and external API integration to enhance system intelligence and functionality.

4.1 BLOCK DIAGRAM

The block diagram represents the high-level operational flow of the system, illustrating how user input is processed and converted into executable actions.



The system consists of the following components:

User Input: The user provides a command through voice or text.

Input Interface: Handles input acquisition and output display.

Gateway Layer: Manages communication using FastAPI and WebSocket for real-time interaction.

AI Processing Layer (Hybrid LLM): Processes input using local or cloud-based LLMs to generate structured intent.

Security Layer: Validates the generated intent against allowlists and blocklists to ensure safe execution.

Execution Engine: Executes validated commands at the operating system level.

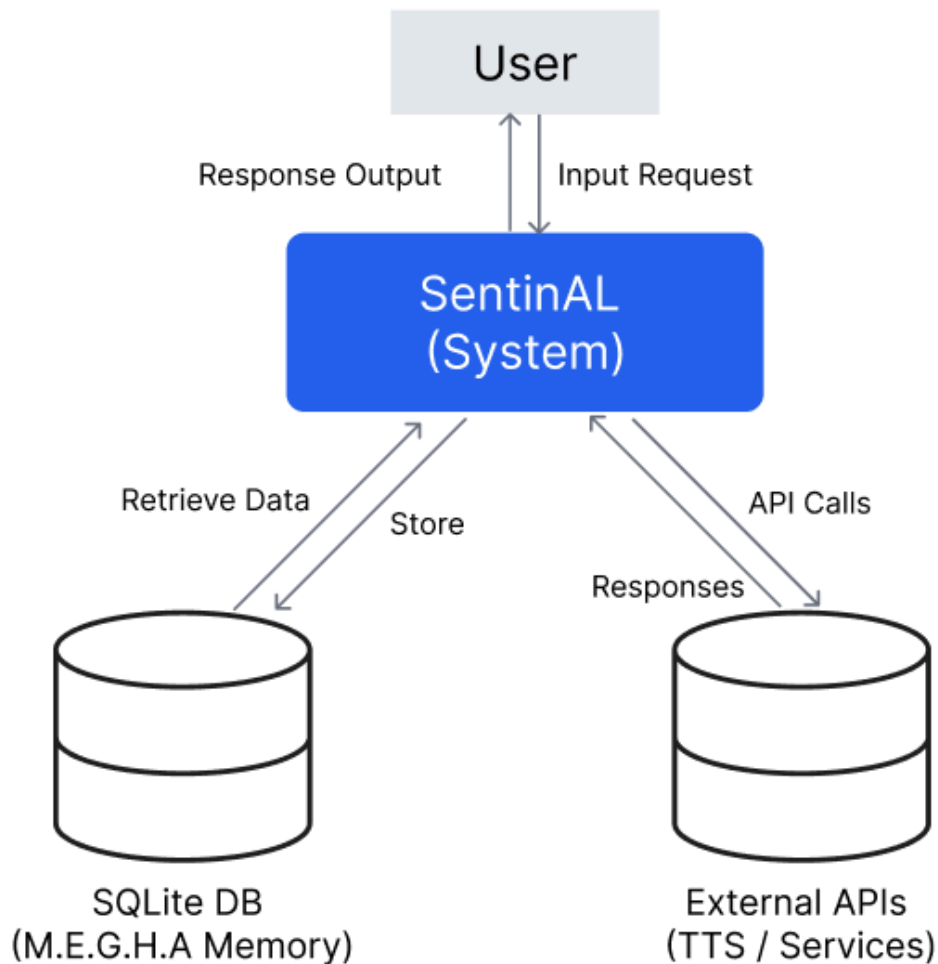
Services Layer: Handles data storage (SQLite), API interactions, and text-to-speech processing.

Output: The feedback loop where execution results are returned to the user.

This layered structure ensures efficient processing, secure execution, and real-time responsiveness.

4.2 DATA FLOW DIAGRAM (DFD)

The Data Flow Diagram (DFD) illustrates how data moves through the system from user input to final output, including intermediate transformations and storage.



4.2.1 DFD Symbols

Standard DFD notations are used:

Process (Circle/Rounded Rectangle): Represents data transformation (e.g., LLM Processing, Execution).

Data Flow (Arrow): Indicates movement of data between components.

Data Store (Open Rectangle): Indicates movement of data between components.

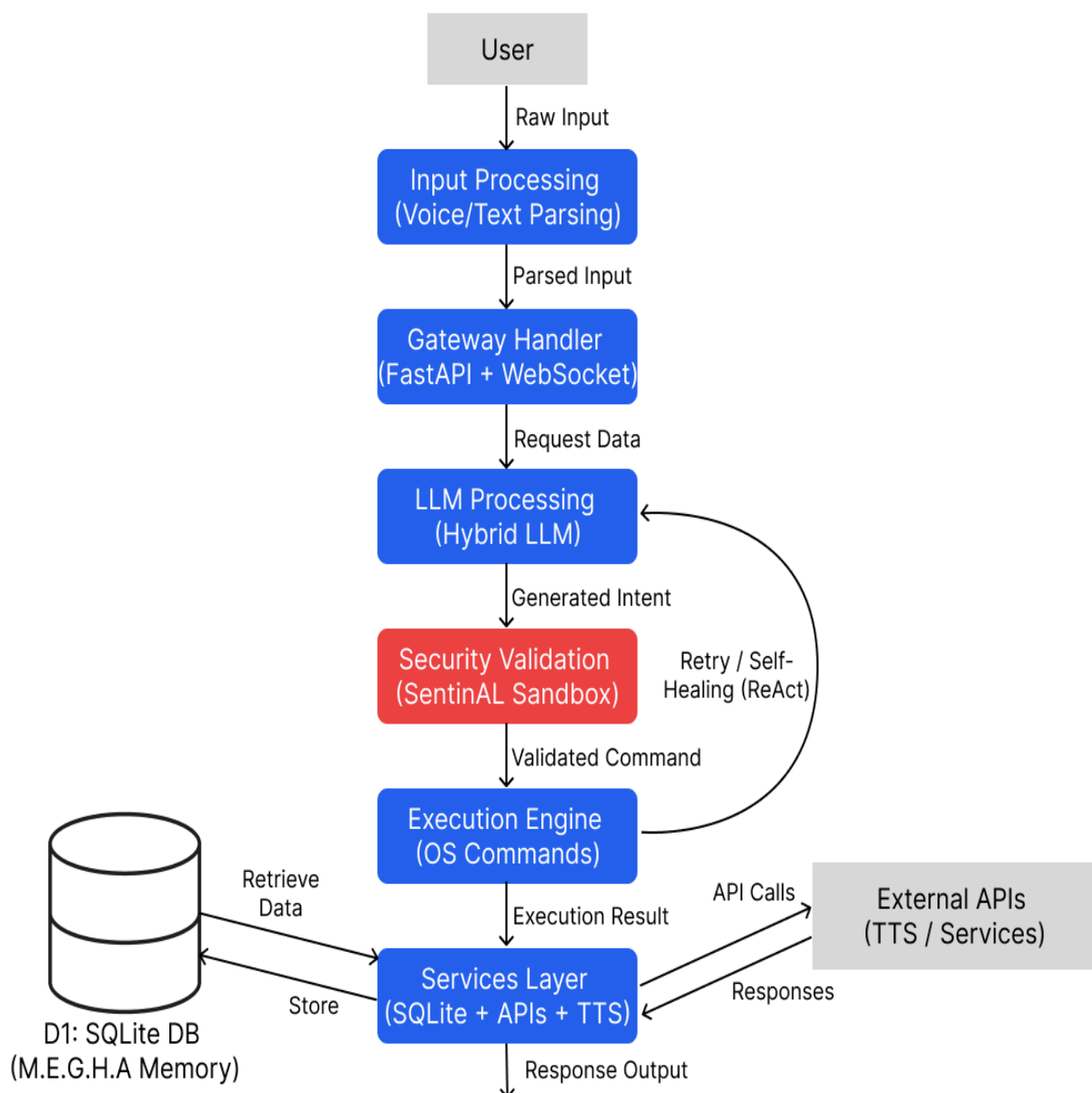
External Entity (Square): Represents external sources/destinations (e.g., User, APIs).

4.2.2 Context Level Diagram

The Level 0 DFD represents SentinAL as a single system interacting with external entities.

- The **User** provides input commands.
- The system processes the input and returns responses.
- The system interacts with **SQLite (M.E.G.H.A memory)** for data storage and retrieval.
- The system communicates with **external APIs** for additional services such as TTS and cloud-based processing.

This diagram provides a high-level overview of system boundaries and interactions.



4.2.3 Level 1 DFD

The Level 1 DFD decomposes the system into internal processes:

1. **Input Processing:** Receives and parses user input (voice/text).
2. **Gateway Handling:** Manages real-time communication via FastAPI and WebSockets.
3. **LLM Processing:** Uses hybrid AI (local + cloud models) to generate structured intent.
4. **Security Validation:** Ensures commands comply with security policies using the SentinelAL sandbox.
5. **Execution Engine:** Executes validated commands on the host operating system.
6. **Services Layer:** Handles memory storage (SQLite), API interactions, and response generation.

Additionally, the system incorporates a **ReAct (Reasoning + Acting) loop**, which enables self-healing by retrying failed executions with improved reasoning.

4.3 UNIFIED MODELING LANGUAGE (UML)

UML diagrams are used to represent both structural and behavioral aspects of the system.

4.3.1 Class Diagram

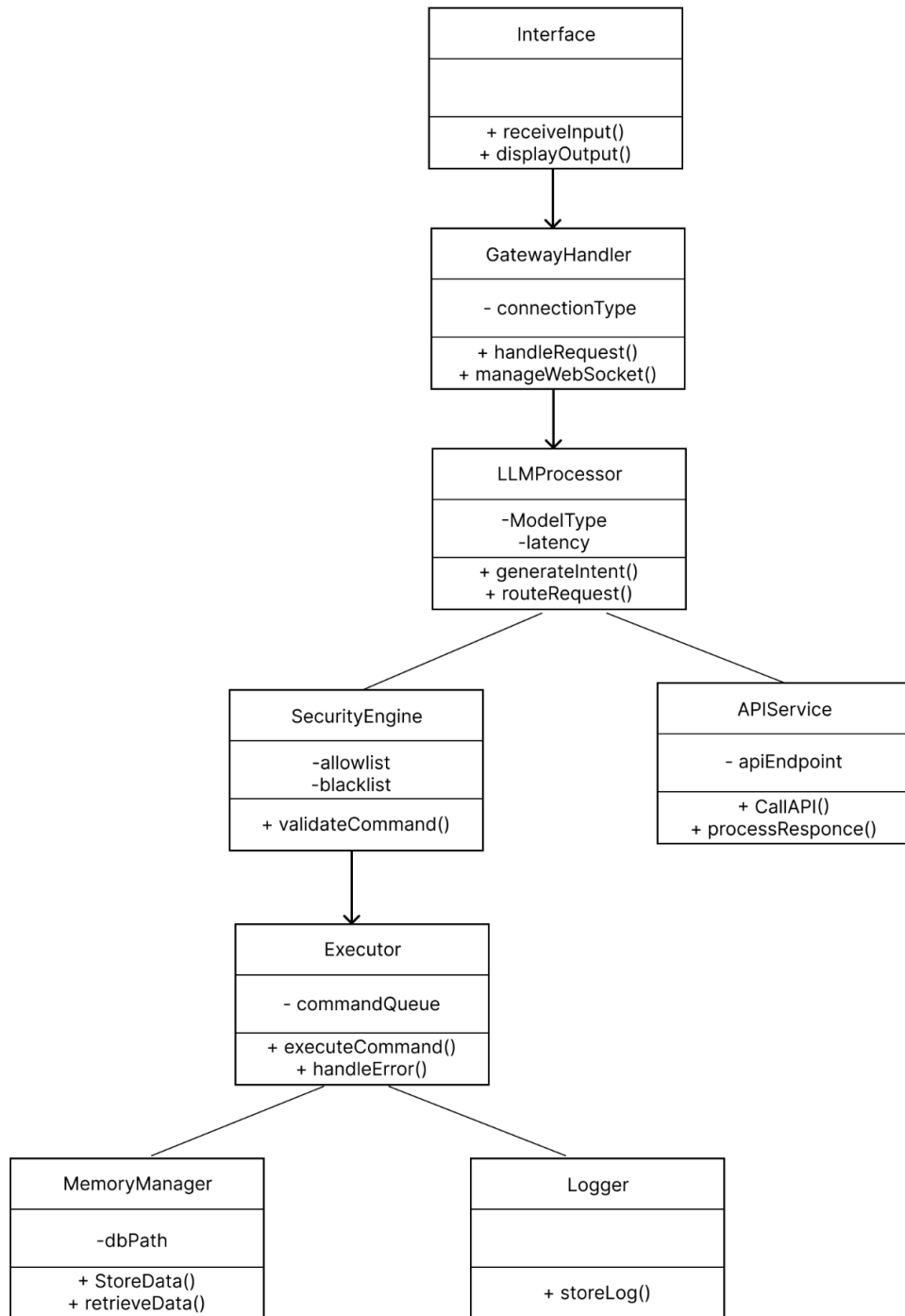
The class diagram represents the static structure of the system and defines relationships between components.

Key classes include:

- **Interface:** Handles user interaction (methods: `receiveInput()`, `displayOutput()`).
- **GatewayHandler:** Manages communication and request routing.
- **LLMProcessor:** Processes input using hybrid AI models (methods: `generateIntent()`, `routeRequest()`).
- **SecurityEngine:** Validates commands using allowlists and blocklists (methods: `validateCommand()`).
- **Executor:** Executes system-level commands (methods: `executeCommand()`, `handleError()`).

- **MemoryManager (SQLite):** Stores and retrieves data (methods: `storeData()`, `retrieveData()`).
- **APIService:** Handles external API communication (methods: `callAPI()`, `processResponse()`).
- **Logger:** Records system activity and errors.

These classes interact to ensure secure and efficient execution of commands.



4.3.2 Use Case Diagram

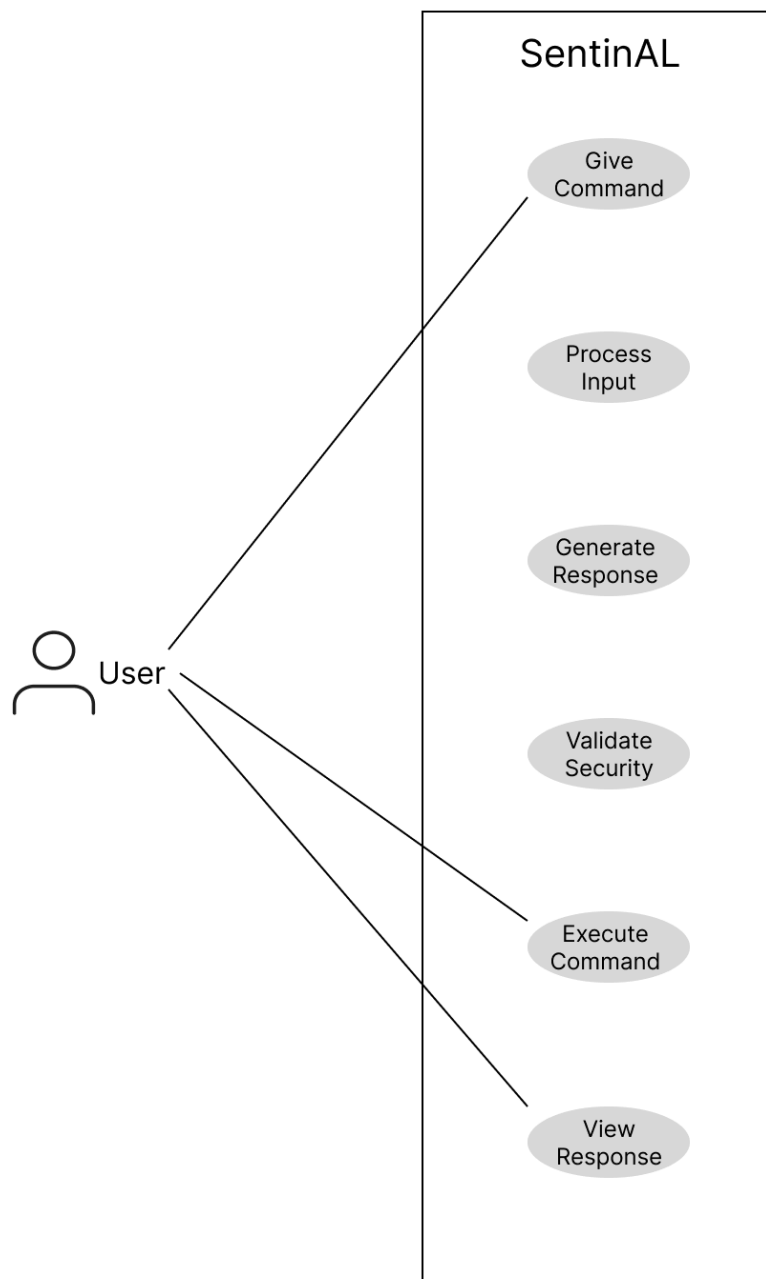
The use case diagram represents the interaction between the user and the system.

The primary actor is the **User**, who interacts with the system to:

- Provide commands

- Execute tasks
- Receive responses

The system internally performs actions such as input processing, AI-based intent generation, security validation, and command execution. The diagram emphasizes user-visible functionalities rather than internal processes.



4.3.3 Sequence Diagram

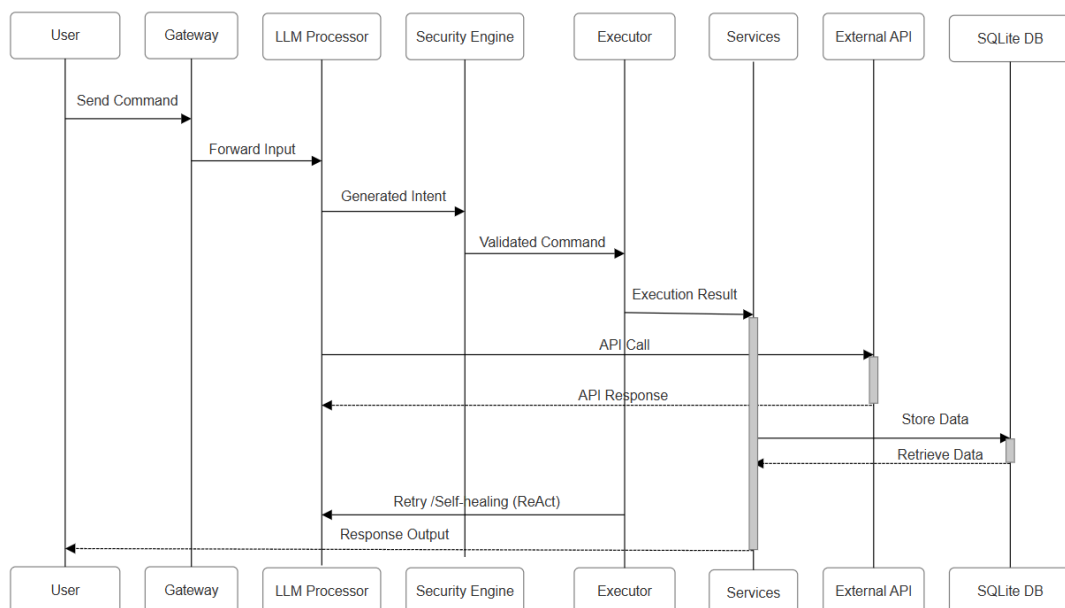
The sequence diagram illustrates the chronological interaction between system components during execution.

The flow includes:

- User sends a command to the Gateway.
- Gateway forwards input to the LLM Processor.
- LLM generates intent and sends it to the Security Engine.
- Security Engine validates the command.
- Executor executes the validated command.
- Services layer handles API calls and database operations.
- Response is returned to the user.

The diagram also includes:

- **API interactions** for cloud services
- **Database operations** for memory storage
- **ReAct loop** for self-healing execution



4.3.4 Activity Diagram

The activity diagram represents the workflow of the system, including decision-making and control flow.

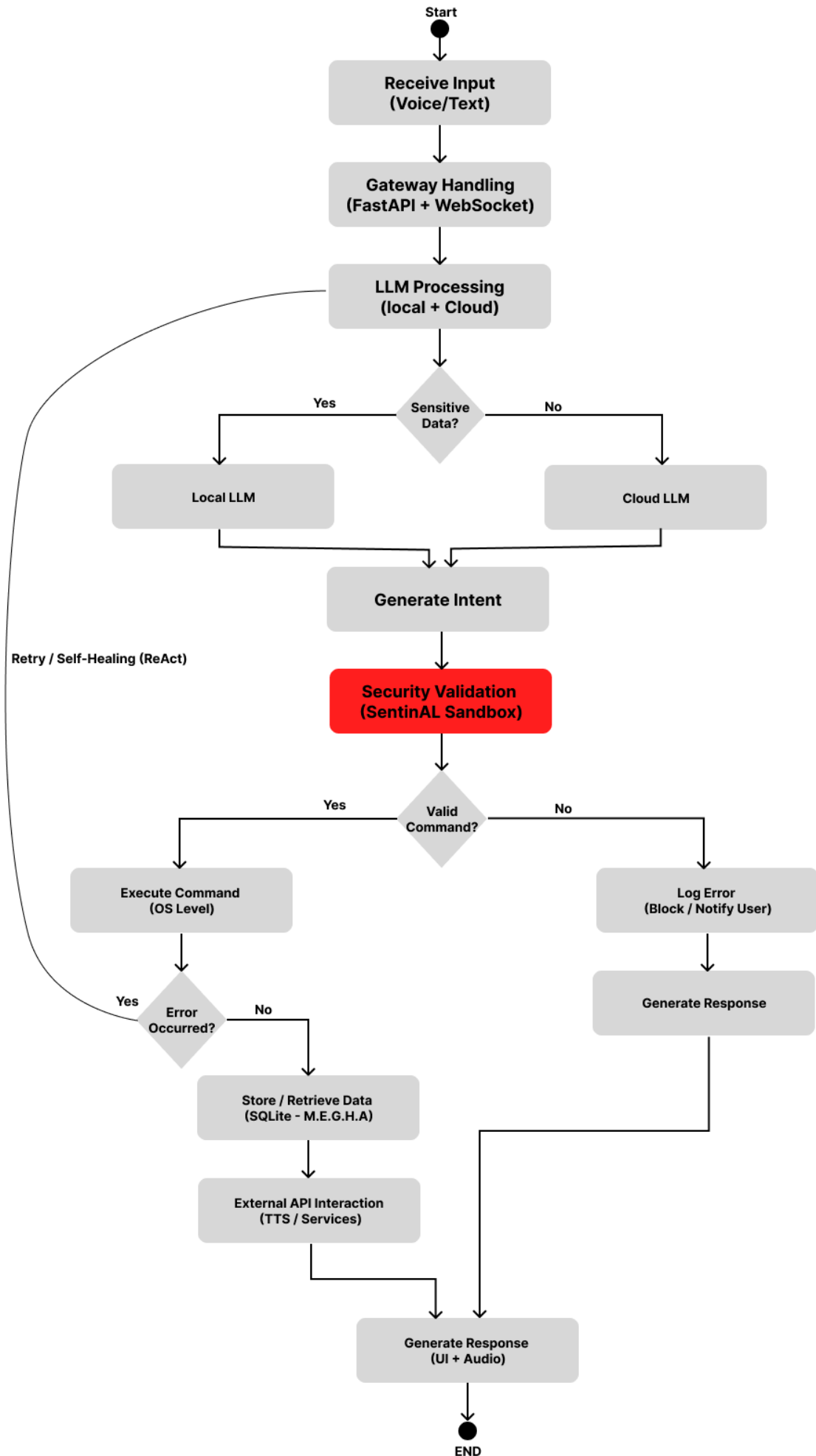
The process includes:

- Receiving user input
- Gateway communication
- Hybrid LLM processing (local or cloud based on sensitivity)
- Intent generation
- Security validation
- Command execution
- Error handling using ReAct self-healing loop
- Data storage and API interaction
- Response generation and output

Decision nodes ensure proper branching based on:

- Data sensitivity
- Command validity
- Execution errors

This diagram highlights the intelligent and adaptive behavior of the system.



5.0 IMPLEMENTATION

The implementation phase focuses on transforming the system design into a fully functional software solution. It involves integrating various components such as user interfaces, backend processing modules, artificial intelligence models, and system-level execution mechanisms into a unified architecture.

For SentinAL, the implementation emphasizes modularity, real-time communication, and secure execution. The system is designed using an asynchronous programming approach to ensure that different processes such as input handling, command processing, and execution can operate simultaneously without affecting overall system performance. This allows the system to maintain responsiveness even during complex multi-step operations.

The implementation also ensures that all system components are loosely coupled, making it easier to update or extend individual modules without impacting the entire system.

5.1 OVERVIEW

The implemented system follows a layered architecture in which each module performs a specific role within the overall workflow. The system begins by capturing user input through a graphical interface, which supports both text and voice-based commands. This input is transmitted to the backend using WebSocket communication, enabling continuous and real-time interaction between the user interface and the processing engine.

Once the input is received, it is processed by the system using language models that interpret the user's intent and convert it into structured commands. These commands are then passed through a validation layer that ensures all operations are safe and permitted before execution.

After validation, the execution module performs the required operations on the operating system. The results of these operations are sent back to the user interface, where they are displayed in real time. This continuous feedback loop enhances user experience and ensures transparency in system behavior.

5.2 TOOLS AND TECHNOLOGIES USED

The system is implemented using a combination of modern technologies that support real-time processing, artificial intelligence integration, and efficient system interaction.

Python 3.10 is used as the primary programming language due to its extensive support for AI development and system-level operations. FastAPI is used as the backend framework because

it provides high-performance asynchronous capabilities and supports WebSocket communication for real-time data exchange.

The frontend interface is developed using React, which enables the creation of a dynamic and responsive user interface. SQLite is used as the database for storing system memory, indexed file paths, and execution logs due to its lightweight and efficient nature.

For artificial intelligence processing, the system integrates both local and cloud-based models. Ollama is used for running local language models, ensuring privacy for sensitive tasks, while cloud APIs such as Groq and Gemini are used for handling more complex operations that require higher computational power.

These technologies work together to provide a stable, efficient, and scalable implementation of the system.

5.3 SYSTEM ARCHITECTURE IMPLEMENTATION

The system architecture is implemented using an asynchronous and event-driven approach. The FastAPI server acts as the central communication layer, maintaining a persistent WebSocket connection with the frontend. This allows the system to continuously exchange data without relying on repeated HTTP requests.

When a user submits a command, the system processes it without blocking the main execution flow. The input is passed through different layers of the system, including processing, validation, and execution, in a sequential yet non-blocking manner.

This design enables the system to handle multiple tasks simultaneously. For example, while a command is being executed, the system can still provide real-time updates to the user interface. This improves system responsiveness and ensures a smooth user experience during operation.

5.4 MODULE IMPLEMENTATION

The system is divided into multiple modules, each responsible for a specific function within the overall workflow.

The Gateway Module manages communication between the frontend and backend using WebSocket connections. It ensures reliable data transmission and handles user requests efficiently.

```
# server.py - Gateway Module Implementation
```

```

from fastapi import FastAPI, WebSocket, WebSocketDisconnect
import asyncio

from core.processor import HybridRouter

app = FastAPI(title="SentinAL Gateway")
router = HybridRouter()

@app.websocket("/ws/agent")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()

    await websocket.send_json({"status": "CONNECTED", "layer": "Gateway"})

    try:
        while True:
            # Receive User Command

            user_input = await websocket.receive_text()

            await websocket.send_json({"telemetry": f"[INFO] Received: {user_input}"})

            # Pass to Processing Module

            intent_payload = await router.process_intent(user_input,
websocket)

            # Stream final status back to UI

            await websocket.send_json({"status": "COMPLETED", "payload":
intent_payload})

    except WebSocketDisconnect:

        print("[WARNING] Client Disconnected. Closing pipeline.")

```

The Processing Module interprets user input using language models and converts it into structured commands that can be executed by the system.

```

# processor.py - Processing Module (Hybrid Router)

import re

from api.groq_client import query_groq
from api.ollama_client import query_local_ollama

class HybridRouter:

```

```

def __init__(self):
    # Keywords that trigger offline privacy mode
    self.sensitive_keywords = [r"tax", r"password", r"private",
r"financial", r"personal"]

    async def process_intent(self, user_text: str, websocket):
        # Evaluate Privacy Sensitivity

        is_sensitive = any(re.search(pattern, user_text.lower()) for pattern
in self.sensitive_keywords)

        if is_sensitive:
            await websocket.send_json({"telemetry": "[SECURITY] Routing to
Local Ollama..."})

            response = await query_local_ollama(user_text) # Offline
Execution

        else:
            await websocket.send_json({"telemetry": "[INFO] Routing to Cloud
Groq API..."})

            response = await query_groq(user_text) # Cloud Execution

        return response

```

The Security Module validates all commands before execution. It checks for unsafe operations using predefined rules such as allowlists and blocklists to ensure system safety.

```

# validator.py - Security Module Sandbox

COMMAND_BLOCKLIST = ["format", "rmdir", "del", "diskpart", "regedit"]
RESTRICTED_PATHS = ["C:\\Windows\\System32", "C:\\Windows", "C:\\Program
Files"]

def validate_payload(ai_json_intent: dict) -> bool:
    action = ai_json_intent.get("action", "").lower()
    target_path = ai_json_intent.get("target_path", "")

    # 1. Check Command Blocklist

    for blocked_cmd in COMMAND_BLOCKLIST:
        if blocked_cmd in action:
            raise Exception("SentinAL Halt: Destructive command detected.")

```

```

# 2. Check Path Sandbox

for restricted in RESTRICTED_PATHS:
    if restricted.lower() in target_path.lower():
        raise Exception("SentinAL Halt: Unauthorized directory access.")

    return True

The Execution Module performs the validated commands on the operating system
using Python libraries such as subprocess and os. It ensures that tasks are
executed correctly and efficiently.

# executor.py - Execution Module (with Self-Healing)

import subprocess

from core.processor import HybridRouter

def execute_with_react(command: str, max_retries: int = 2):
    attempts = 0
    current_command = command

    while attempts < max_retries:
        try:
            # Execute Native OS Command

            result = subprocess.run(current_command, shell=True,
capture_output=True, text=True, check=True)

            return result.stdout # Success

        except subprocess.CalledProcessError as error:
            # Catch the Error and Self-Heal

            error_message = error.stderr

            current_command = HybridRouter.heal_syntax(current_command,
error_message)

            attempts += 1

    return "CRITICAL: Execution failed after maximum self-healing retries."

```

The Services Module handles additional functionalities such as database operations, logging, and text-to-speech processing.

Each module operates independently, which improves system maintainability and allows for future enhancements.

5.5 WORKFLOW EXECUTION

The workflow of the system begins when the user provides input through text or voice. This input is transmitted to the backend through the WebSocket connection.

The system processes the input and generates structured commands using language models. These commands are then validated by the security layer to ensure that they do not violate any system constraints.

Once validated, the execution module performs the required operations. During execution, the system continuously sends updates back to the user interface, providing real-time feedback on the status of the task.

If an error occurs during execution, the system attempts to correct it and continue the workflow. This improves reliability and reduces the need for manual intervention.

The workflow ensures efficient, secure, and continuous operation of the system.

5.6 KEY FEATURES IMPLEMENTED

The implemented system includes several key features that enhance its functionality and performance.

Hybrid AI Routing enables the system to dynamically switch between local and cloud-based language models based on task requirements. This ensures both privacy and performance.

Real-Time Communication is achieved through WebSocket integration, allowing continuous interaction between the user interface and backend system.

Security Validation ensures that all commands are checked before execution, preventing unsafe or unauthorized operations.

Self-Healing Execution allows the system to detect errors during execution and attempt automatic correction, improving reliability.

Memory-Based Path Resolution uses a database to store and retrieve accurate file paths, ensuring correct execution of file-related tasks.

These features collectively contribute to a robust and efficient system implementation.

6.0 TESTING

Testing is a critical phase in the software development process that ensures the system operates correctly, efficiently, and securely. For SentinAL, testing was conducted to validate

the functionality of each module, verify system performance, and ensure that all security mechanisms operate as intended.

Since the system integrates artificial intelligence with operating system-level execution, special care was taken to test the accuracy of command interpretation, correctness of execution, and safety of operations. The testing phase also ensured that the system performs reliably under different usage conditions and provides consistent results.

6.1 INTRODUCTION

The testing process was designed to evaluate both functional and non-functional aspects of the system. Functional testing ensured that the system correctly interprets user inputs and executes commands as expected. Non-functional testing focused on system performance, responsiveness, and reliability.

Different types of testing were carried out to ensure that all components of the system work independently and together as a complete system. These include unit testing, integration testing, and system testing.

6.1.1 Unit Testing

Unit testing was performed to verify the functionality of individual modules within the system. Each component was tested independently to ensure it performs its intended task correctly.

Modules such as input processing, database operations, and communication handling were tested separately. For example, database queries were tested to ensure accurate retrieval of file paths, and WebSocket communication was tested for correct message transmission.

This level of testing helped in identifying and fixing errors at an early stage.

6.1.2 Integration Testing

Integration testing was conducted to verify the interaction between different modules of the system. It ensured that data flows correctly from one module to another without errors.

For example, the connection between the input module, processing module, validation layer, and execution module was tested to ensure smooth communication. This testing confirmed that the system components work together as expected.

6.1.3 System Testing (Black-Box)

System testing was performed to evaluate the system as a complete unit. In this approach, the system was tested based on user inputs and expected outputs without focusing on internal implementation details.

Various user commands were tested to ensure that the system correctly processes inputs and produces the desired outputs. This type of testing helped in validating the overall functionality and usability of the system.

6.2 DESIGN OF TEST CASES AND SCENARIOS

Test cases were designed to evaluate normal operations, edge cases, and system safety.

In one scenario, the system was given a command to open an application and perform a specific task. The system successfully interpreted the command and executed it correctly.

In another scenario, commands involving local file access were tested to ensure that the system handles sensitive data appropriately. The system correctly processed these commands without exposing data externally.

Error scenarios were also tested by providing incorrect inputs. The system detected errors and attempted to correct them, improving reliability.

Additionally, unsafe commands were tested to verify the effectiveness of the security layer. The system successfully blocked such commands and prevented execution.

Test Case ID	Input Command	Expected Output	Actual Output	Status
TC-01	Open Chrome	System launches Chrome successfully	As Expected	Pass
TC-02	Show my downloads	System opens Downloads folder	As Expected	Pass
TC-03	Search for my project file	System returns matching file	As Expected	Pass

TC-04	Delete system folder	Validation Module blocks command and shows "Access Denied"	As Expected	Pass
TC-05	Launch Notepad	System opens Notepad	As Expected	Pass
TC-06	Empty Input	System prompts user to enter valid command	As Expected	Pass

6.3 INPUT TESTING

Input testing was conducted to ensure that the system correctly handles different types of user inputs. Both text and voice inputs were tested under various conditions.

The system was tested with valid commands, incomplete inputs, and ambiguous phrases to evaluate its ability to interpret user intent. The results showed that the system can process natural language inputs effectively and convert them into structured commands.

This testing ensured that the system is user-friendly and capable of handling real-world inputs.

6.4 EXECUTION TESTING

Execution testing was performed to verify that the system executes validated commands correctly on the operating system.

Various tasks such as opening applications, accessing files, and retrieving information were tested. The system successfully executed these operations and provided real-time feedback to the user.

This testing confirmed that the execution module works reliably and integrates properly with the operating system.

6.5 ERROR HANDLING

Error handling was tested to ensure that the system can detect and manage failures during execution.

When incorrect or invalid commands were provided, the system identified the errors and attempted to correct them. In cases where execution was not possible, the system safely terminated the process and provided appropriate feedback to the user.

This mechanism improves system stability and prevents unexpected failures.

6.6 PERFORMANCE TESTING

Performance testing was conducted to evaluate the speed and efficiency of the system.

The system demonstrated fast response times during command processing and execution. Database operations for retrieving file paths were performed efficiently, ensuring minimal delay.

Real-time communication through WebSockets enabled continuous updates, improving user experience. The system maintained stable performance even during multiple operations.

The system maintained consistent performance under repeated execution scenarios, demonstrating its reliability for continuous usage.

6.7 TEST RESULTS SUMMARY

The testing phase confirmed that the system meets both functional and non-functional requirements.

All major functionalities, including input processing, validation, execution, and feedback, were tested successfully. The system demonstrated reliable performance, accurate execution, and effective security validation.

Overall, the results indicate that the system is stable, efficient, and suitable for real-world desktop automation tasks.

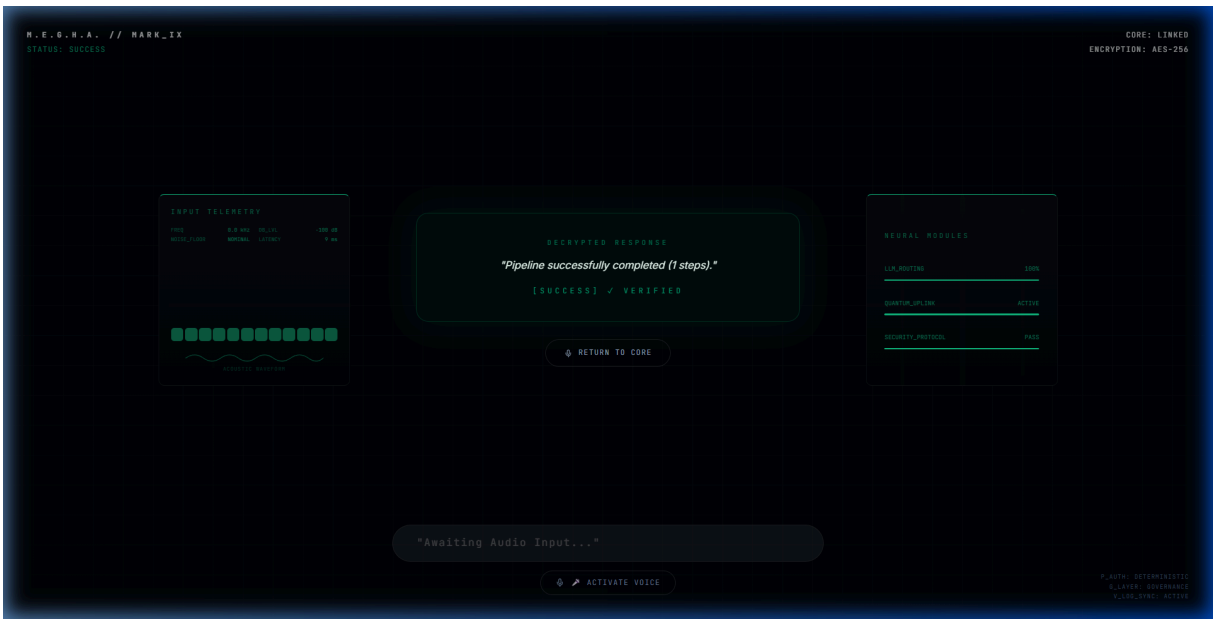
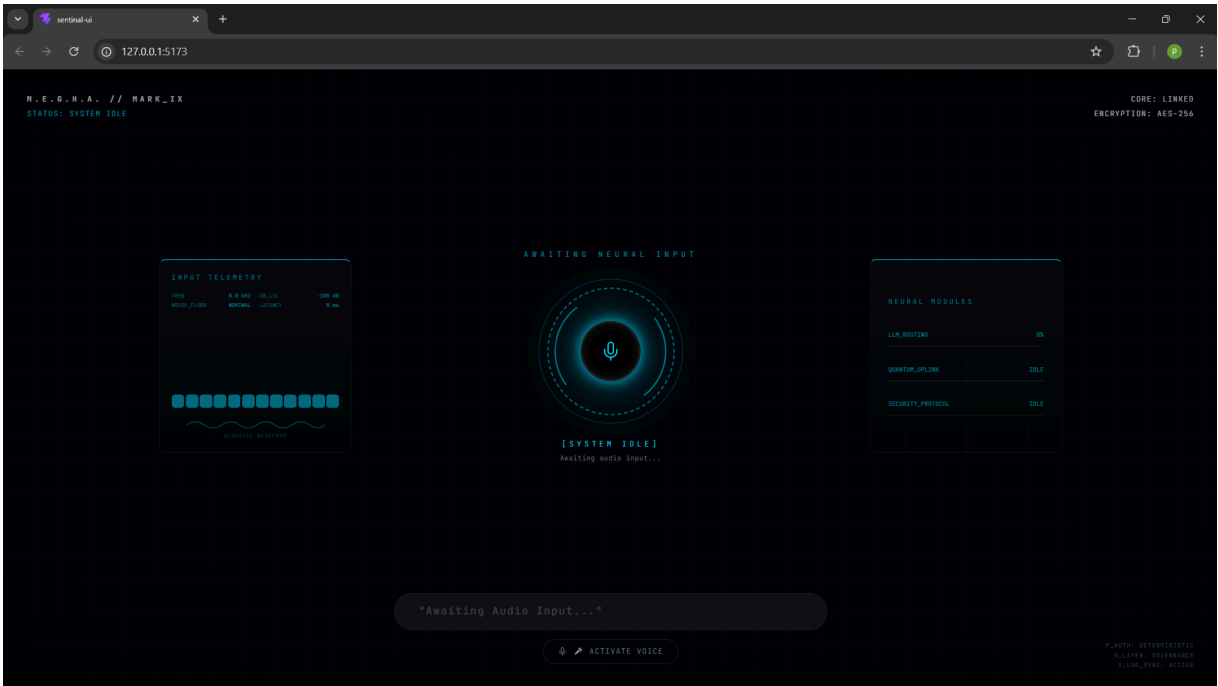
7.0 OUTPUT SCREENS

The output screens section presents the visual results of the system during execution. The interface updates dynamically to display system states such as input processing, validation, execution, and output generation. Screenshots are included to demonstrate key functionalities of the system.

7.1 SYSTEM INITIALIZATION AND DASHBOARD

This screen displays the initial state of the React frontend upon launching the application. The interface shows an active WebSocket connection status indicator (displaying "Connected" in

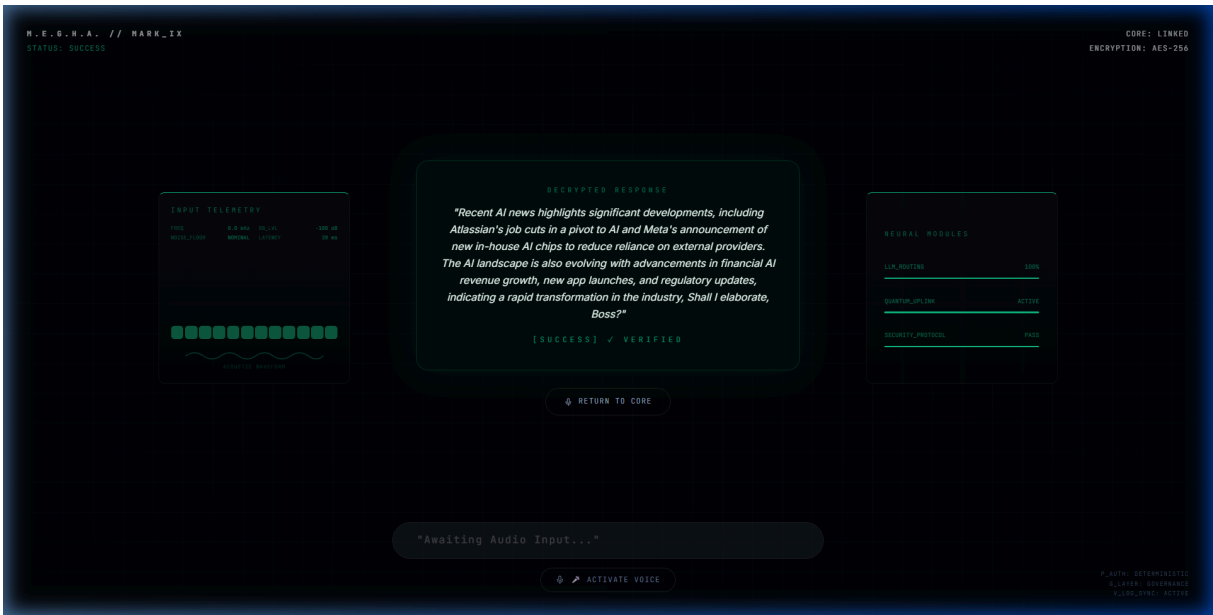
green). The dashboard features a clean input field for text commands, a microphone toggle for voice activation featuring Acoustic Echo Cancellation, and an empty telemetry console awaiting the first execution.



7.2 SUCCESSFUL MULTI-STEP CLOUD EXECUTION

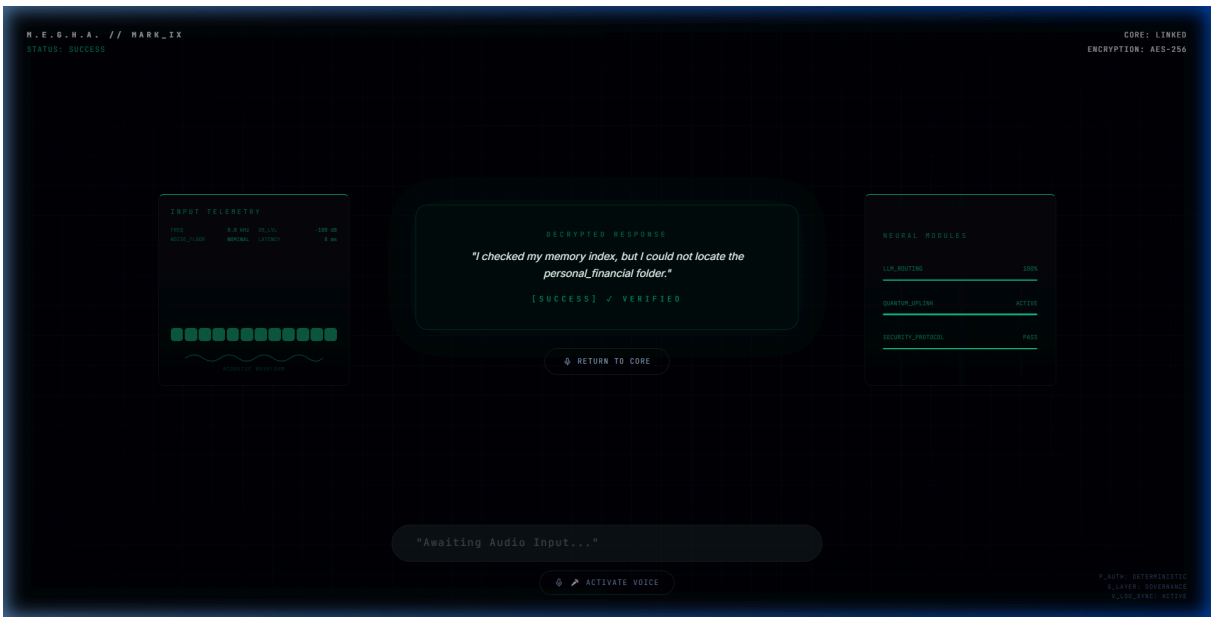
This screen captures the system successfully completing a generalized orchestration task, such as "Open the browser and search for the latest AI news." The telemetry console explicitly shows the routing decision ("Routing to Cloud: Groq Llama-3.3"), the generated JSON array

of intents, the successful SentinAL validation check, and the final operating system execution status. A "Task Completed" notification is visible in the output log.



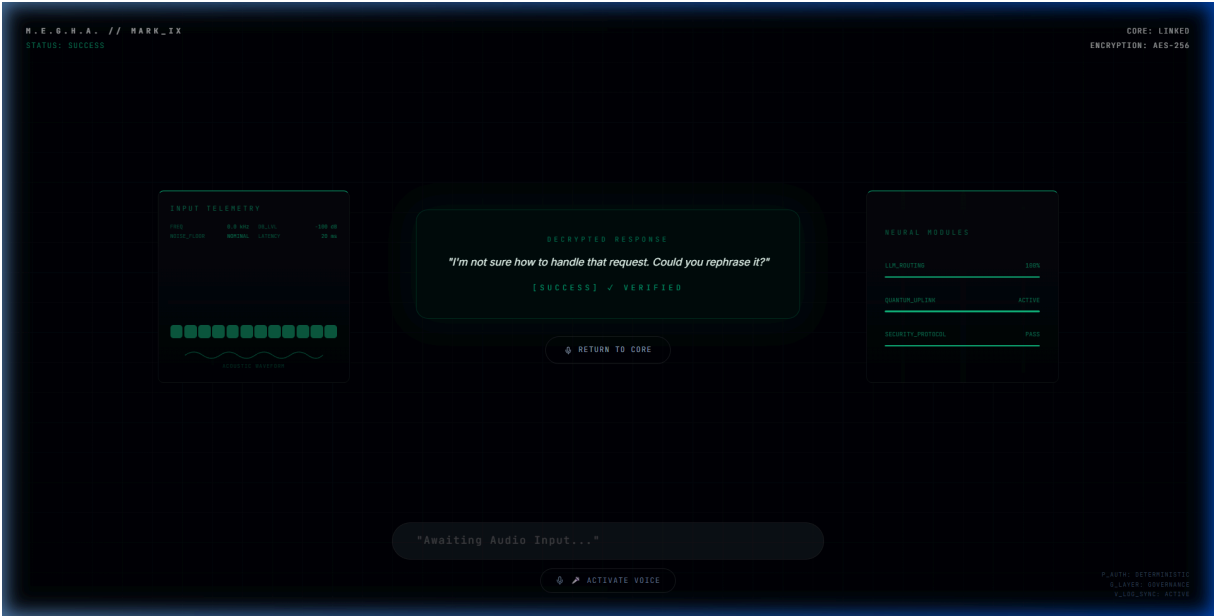
7.3 PRIVACY-ROUTED LOCAL EXECUTION

This screenshot highlights the condition-based hybrid routing in action. The user input contains a sensitive keyword, such as "Search my personal financial folder." The telemetry log clearly indicates the routing shift ("Sensitive Intent Detected: Routing to Local Offline Model - Ollama"). The execution console confirms the successful traversal of the local directory without establishing external cloud network connections.



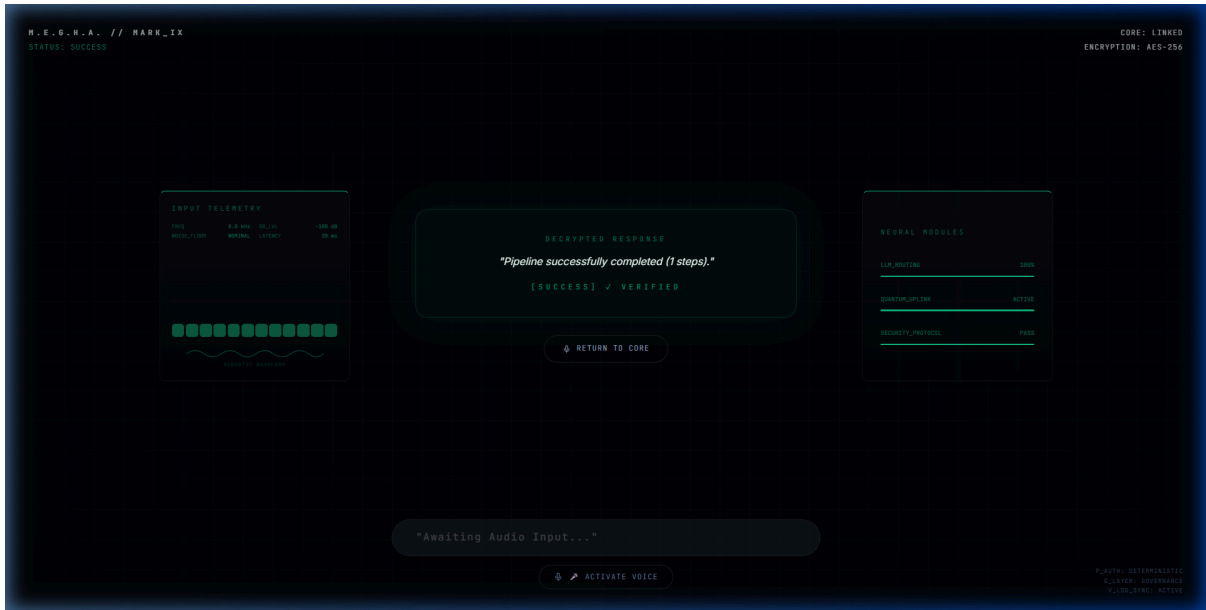
7.4 SENTINAL SECURITY BLOCKLIST INTERCEPTION

This image demonstrates the critical SentinAL governance layer actively protecting the host operating system. The user intentionally inputs a malicious command, such as "Format the primary C drive" or "Delete System32." The screen shows a stark red warning within the telemetry console. The log explicitly states "Execution Aborted: Command matches internal blocklist." The system halts entirely, proving the effectiveness of the sandbox validation.



7.5 REACT SELF-HEALING LOOP RECOVERY

This screen illustrates the fault-tolerance mechanism of the execution engine. The user requests an application to open, but the initial language model output contains a minor syntax error (e.g., executing "notepad.exe" instead of "notepad.exe"). The telemetry console displays the captured Windows standard error. Immediately below, the console shows the ReAct loop triggering ("Self-Healing Initiated: Querying LLM for correction"). The final log entry shows the corrected command executing successfully, preventing a fatal crash.



8.0 CONCLUSION

This project successfully demonstrates the integration of natural language processing with operating system-level execution. The SentinAL system enables users to perform desktop operations using simple commands while maintaining system safety and reliability.

The layered architecture ensures modularity and efficient communication between components. The use of a memory-based path resolution mechanism improves execution accuracy, while the security validation layer prevents unsafe operations. The hybrid routing approach balances performance and data privacy by utilizing both local and cloud-based processing.

Overall, the system provides an effective solution for intelligent desktop automation and meets the objectives defined in the project.

9.0 FUTURE ENHANCEMENT

While the current system architecture is stable and functional, several areas offer significant potential for future expansion.

The foremost enhancement involves the complete integration of a multi-modal Vision Processing module. By allowing the artificial intelligence to capture and analyze screen states visually, the system could orchestrate complex graphical tasks that lack traditional shell commands, such as navigating legacy software interfaces or interacting with dynamically rendered web applications.

Another substantial improvement involves extending cross-platform compatibility. The current execution actuator is optimized specifically for Windows environments utilizing native libraries. Future iterations could expand the execution mapping to translate user intents into native Linux bash scripts or macOS terminal commands, establishing a universal desktop assistant architecture.

Finally, introducing dynamic Role-Based Access Control (RBAC) would greatly increase the utility of the system in enterprise environments. Future updates could implement multi-user authentication, allowing administrators to configure unique SentinAL security blocklists and allowlists based on the authenticated user profile, ensuring that guest users operate under stricter sandboxes than system administrators.

10.0 REFERENCES

- [1] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed., Pearson, 2023.
- [2] Python Software Foundation, "Python 3.10 Documentation," 2024. [Online]. Available: <https://docs.python.org/3/>
- [3] S. White and A. Patel, "Validation Logic and Allowlist Verification in Desktop Automation Software," International Conference on Software Security, pp. 88-102, 2023.
- [4] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," International Conference on Learning Representations (ICLR), 2023.
- [5] M. Harrison, "Deploying Large Language Models at the Edge: Performance and Privacy Trade-offs," IEEE Transactions on Edge Computing, vol. 5, no. 2, pp. 45-58, 2024.
- [6] FastAPI and WebSockets Documentation, "Concurrency and async / await," Tiangolo, 2024. [Online]. Available: <https://fastapi.tiangolo.com/>
- [7] Ollama API Documentation, "Local LLM Runtime Optimization," 2024. [Online]. Available: <https://github.com/ollama/ollama/blob/main/docs/api.md>